

Security Audit

DEDCAT (DedcatBounce)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	24
Conclusion	25
Our Methodology	26
Disclaimers	28
About Hashlock	29

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The DEDCAT team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

DedcatBounce is a last-player-standing game on HyperEVM. Players spend \$HYPE to Bounce the cat and reset a live countdown timer. When the timer inevitably hits zero, the last player to Bounce wins the prize pool.

Each Bounce also fuels a referral system, where active players earn rewards from the Bounces made by others they referred into the game. This creates a network of competitive, self-propelling communities known as Bounce Cabals, driving both engagement and growth.

DedcatBounce is the first dApp in the Dedcat ecosystem — a composable treasury of on-chain games and DeFi primitives, all routing revenue back to \$DEDCAT stakers. It serves as the ignition point for Dedcat's flywheel, proving how social gameplay and real yield can power a self-sustaining community owned on-chain economy.

Project Name: DedcatBounce

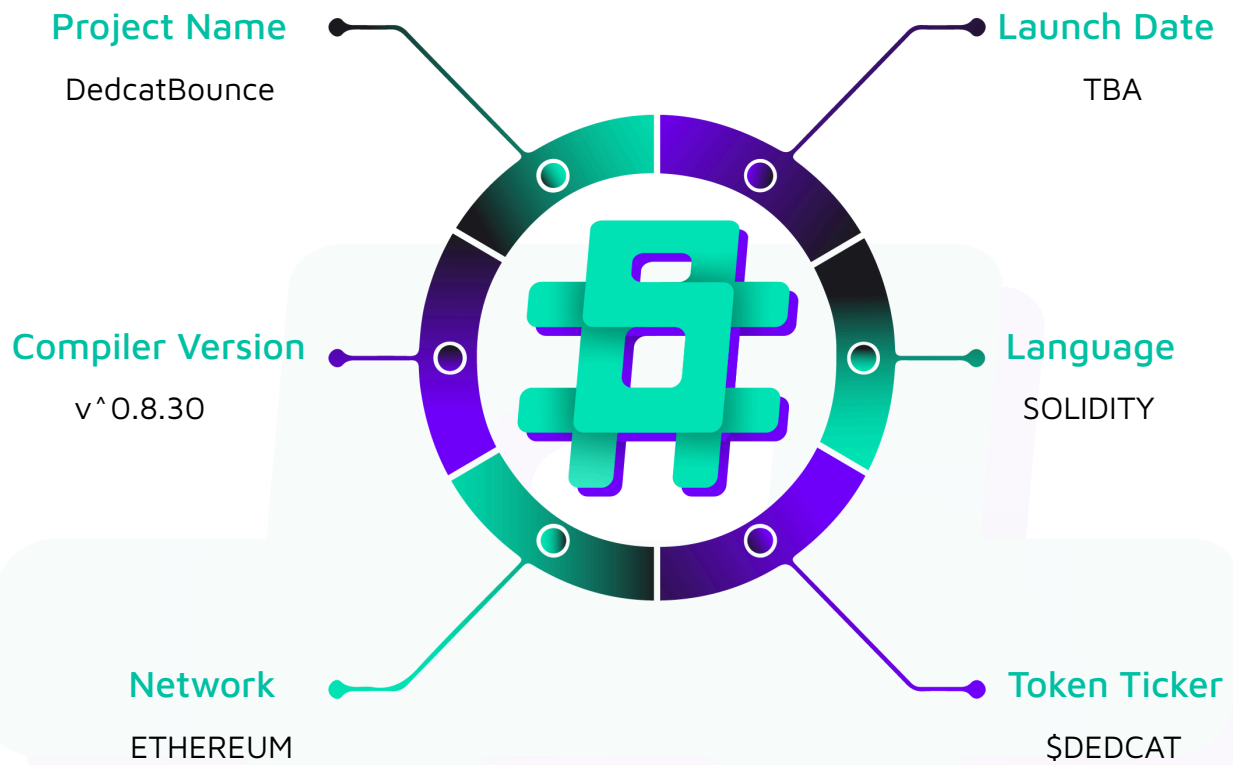
Project Type: dApp

Compiler Version: ^0.8.30

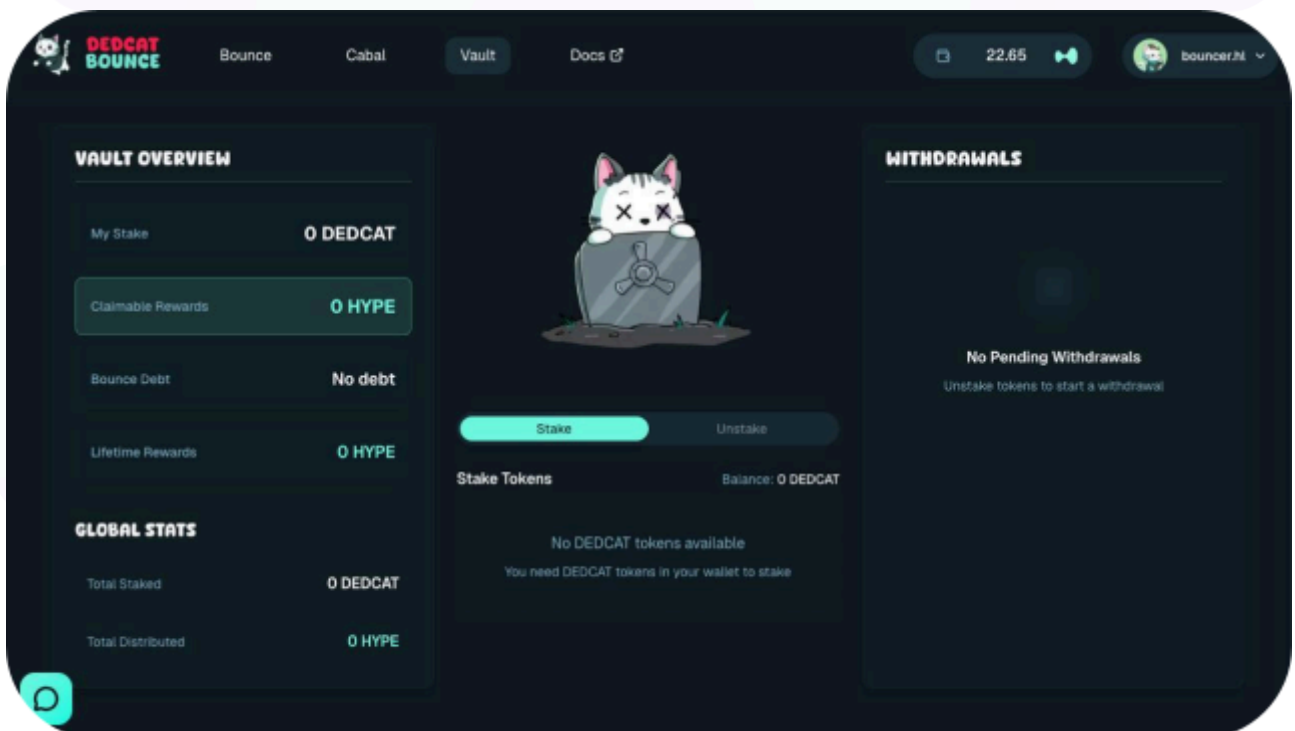
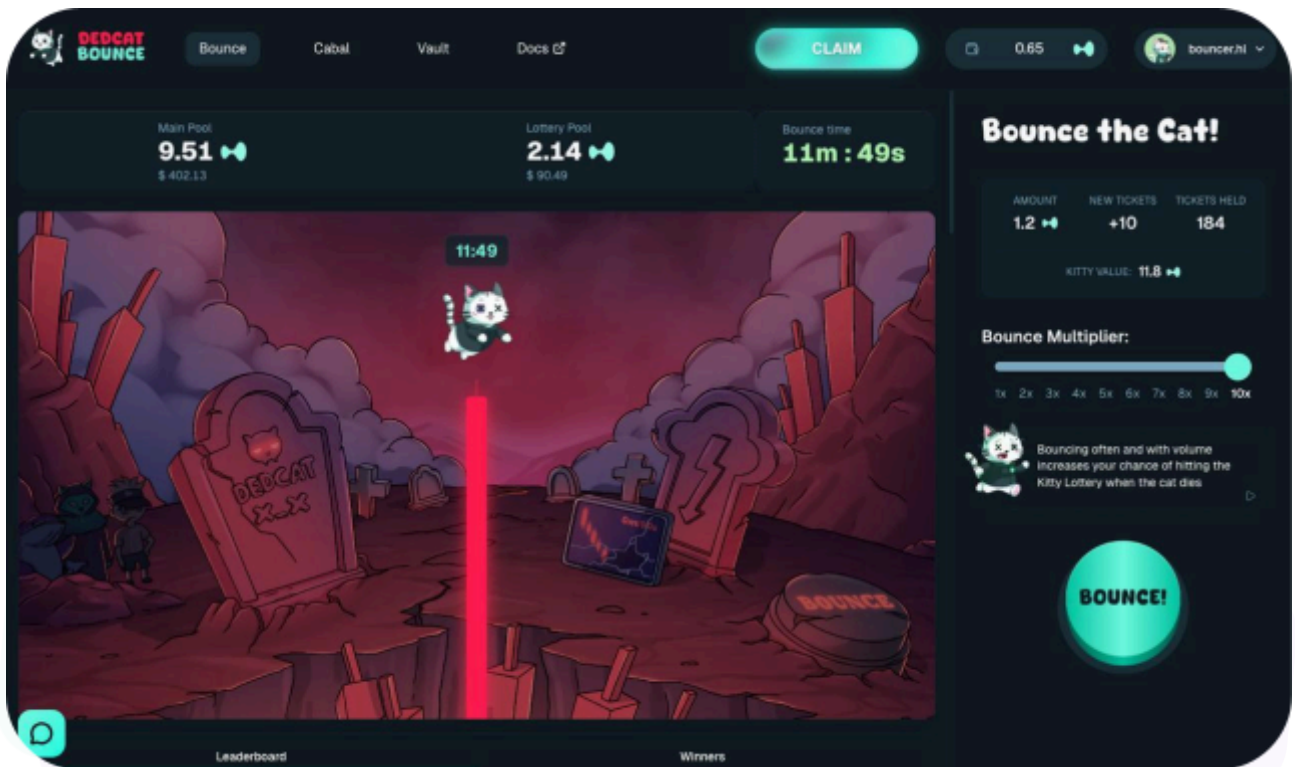
Website: ded.cat

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the DedcatBounce project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	DedcatBounce Smart Contracts
Platform	Ethereum / Solidity
Audit Date	September, 2025
Contract 1	AbstractActivityRecorder.sol
Contract 2	DeadcatBounce.sol
Contract 3	DeadcatCabal.sol
Contract 4	DedcatVault.sol
Contract 5	KittyLottery.sol
Audited GitHub Commit Hash	7aad0fd6df4796f99407e6dce3cd3e7d6ba5b2e1
Fix Review GitHub Commit Hash	c61ca410ba9ae74e749a2436a9d105fc6044352e

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section, and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 High severity vulnerabilities

2 Low severity vulnerabilities

3 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
AbstractActivityRecorder.sol - Tracks per-user activity timestamps and debt balances for inactivity penalties. - Caps accrued debt and credit within fixed thresholds. - Lets inheriting contracts pause debt accrual and record manual activity clears.	Contract achieves this functionality.
DeadcatBounce.sol - Runs the core bounce game: collects entries, updates pots, and determines winners. - Allocates bounce proceeds across cabal, vault, big blind, and lottery components. - Enforces pausing, eligibility checks, and round restarts with entropy-backed randomness.	Contract achieves this functionality.
DeadcatCabal.sol - Maintains a five-level referral tree with activity gating and transparent cabals. - Distributes cabal rewards to active referrers or forwards them to the vault when inactive. - Allows linked players to claim accumulated referral earnings via the bounce contract.	Contract achieves this functionality.
DedcatVault.sol - Offers staking for the DEDCAT ERC20 with	Contract achieves this functionality.

reward accrual in native HYPE. - Enforces withdrawal timelocks and integrates activity-based debt tracking. - Accepts reward top-ups, distributing them pro rata to current stakers.	
KittyLottery.sol - Manages per-round ticket issuance and pot notifications from the bounce contract. - Requests entropy, draws lottery winners, and tracks claimable winnings per user. - Lets the bounce contract mark winnings as claimed during pot payout.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Dedcat project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Dedcat project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] KittyLottery#_draftWinners - Lottery Winner Selection Bias

Description

The lottery uses `randomDataNumber % nrOfBouncers` to select winners, but this creates modulo bias.

Vulnerability Details

```
uint256 randomDataNumber = uint256(  
    keccak256(abi.encodePacked(randomNumber, i))  
);  
  
uint256 winningBouncerIndex = randomDataNumber % nrOfBouncers;
```

If `nrOfBouncers` doesn't divide evenly into 2^{256} , some bouncers have slightly higher probability of winning. An example to illustrate it more clearly:

- If `nrOfBouncers` = 3
- Possible values: 0, 1, 2, 0, 1, 2, 0, 1, 2...
- $2^{256} \% 3 = 1$
- Index 0 appears $(2^{256} / 3) + 1$ times
- Index 1 appears $(2^{256} / 3) + 1$ times
- Index 2 appears $(2^{256} / 3)$ times

Impact

Winner selection is biased because the draw applies `random % nrOfBouncers`. Whenever the participant count is not an exact divisor of 2^{256} , some indices receive one extra outcome in the mapping, so certain players have a marginally higher chance of being chosen. The distortion may be small, but it breaks the "equal odds per ticket"

assumption, undermines fairness claims and introduces reputational risk if the lottery is promoted as unbiased.

Recommendation

Replace the direct modulo with rejection sampling (or another unbiased mapping): repeatedly expand the entropy until the value falls below `type(uint256).max - (type(uint256).max % nrOfBouncers)` and then take the modulus, or otherwise bucket the randomness so every bouncer index is hit exactly the same number of times.

Status

Resolved

[H-02] DedcatVault#_addRewards - Vault Pending Rewards Frontrunning Allows Fund Theft

Description

When the vault has zero stakers, rewards accumulate in `pendingRewards`. A malicious actor can stake a tiny amount right (as low as 1 wei) before a large reward is added, capturing all pending rewards disproportionately.

Vulnerability Details

When there are no active stakers (`totalStaked == 0`), all incoming rewards are stored in the `pendingRewards` variable. However, as soon as one user stakes, even with the smallest amount, the entire accumulated reward balance is released to the stakers, which at that point consists solely of the attacker.

```
function _addRewards(uint256 amount, address sender) internal {  
  
    if (amount == 0) revert ZeroValue();  
  
    // Store pending rewards in case we have no stakers  
  
    if (totalStaked == 0) {  
  
        pendingRewards += amount; // <-- Rewards accumulate while no stakers  
  
        emit RewardAdded(amount, sender);  
  
        return;}  
  
    // Add any pending rewards  
  
    uint256 toDistribute = amount;  
  
    if (pendingRewards > 0) {  
  
        toDistribute += pendingRewards; // <-- All pending distributed to current staker  
  
        pendingRewards = 0;  
  
    }  
  
    _distribute(toDistribute, sender);  
  
}
```


Proof of Concept

```
function testPendingRewardsFrontrun() public {  
  
    // 1. Initial state: no stakers, rewards accumulate  
  
    deadcatVault.addReward{value: 100e18}(); // pendingRewards = 100 HYPE  
  
    assertEq(deadcatVault.pendingRewards(), 100e18);  
  
    // 2. Attacker sees next addReward(50) in mempool  
  
    address attacker = address(alice);  
  
    vm.startPrank(attacker);  
  
    // 3. Frontrun: stake a tiny amount  
  
    deadcatToken.approve(address(deadcatVault), 1);  
  
    deadcatVault.stake(1); // totalStaked = 1 wei  
  
    vm.stopPrank();  
  
    // 4. Target tx executes  
  
    deadcatVault.addReward{value: 50e18}();  
  
    // toDistribute = 50 + 100 (pending) = 150 HYPE  
  
    // Attacker owns 100% → receives all rewards  
  
    assertEq(deadcatVault.earned(attacker), 150e18);  
  
    // 5. Attacker claims stolen rewards  
  
    vm.startPrank(attacker);  
  
    deadcatVault.claim();  
  
    vm.stopPrank();  
  
    assertEq(deadcatVault.pendingRewards(), 0);  
  
}
```

Impact

A malicious actor can drain all accumulated pendingRewards with a low amount of stake at the beginning.

Recommendation

Require a minimum stake threshold before rewards can be distributed

Status

Resolved

Low

[L-01] DeadcatCabal#claim - Referral Claim DoS via console.log

Description

DeadcatCabal.claim still calls Foundry's `console.log`, which relies on a cheat code that exists only in the Forge testing environment:

```
import {console} from "forge-std/console.sol";

function claim() external nonReentrant {

    console.log("Claim Sender", msg.sender);

    ...

}
```

On live networks, that call targets an unset address and the EVM aborts, so every claim reverts. It's a placeholder left over from development rather than a security issue, but it will block referral payouts if not removed before deployment.

Recommendation

Drop the debugging import/statement. This keeps test logs if needed while ensuring production builds let claims succeed.

Status

Resolved

[L-02] KittyLottery#claimable, markClaimed - Unbounded Lottery Claim Loop Can Exceed Gas Limits

Description

KittyLottery.claimable() and markClaimed() iterate across the full bouncerRounds[user] array every time they run. Because that array grows indefinitely with each round a user joins, the gas cost of computing a claim rises linearly. After enough rounds (hundreds or thousands), the call could approach chain gas limits, risking failed claim transactions for high-volume participants.

Recommendation

Expose paginated variants (e.g. claimable(user, start, end) / markClaimed(user, start, end)) so callers process a bounded slice per transaction.

Status

Resolved

QA

[Q-01] Contracts - Naming Inconsistency: Dedcat and Deadcat

Description

There are inconsistent uses of the prefixes `Dedcat` and `Deadcat` across filenames, contract names, and comments. This inconsistency creates confusion during development and testing, increases the likelihood of import mistakes, and may lead to deployment of the wrong contract variant.

Recommendation

Choose a single standardized prefix (either “Deadcat” or “Dedcat”) and enforce it across the codebase.

Status

Resolved

[Q-02] DedcatVault#setBounce - Emits Incorrect Event Parameters (Old/New Inversion)

Description

The `setBounce(address _bounce)` function in `DedcatVault.sol` incorrectly assigns the `oldBounce` variable and emits event parameters in the wrong order.

```
function setBounce(address _bounce) external onlyOwner {  
    if (_bounce == address(0)) revert ZeroAddress();  
  
    address oldBounce = _bounce; // ← Should capture previous bounce  
  
    bounce = _bounce;  
  
    emit BounceUpdated(_bounce, oldBounce); // ← Parameters inverted (new, old)  
}
```

As a result, the emitted `BounceUpdated` event logs (new, new) values instead of (old, new), leading to inaccurate or misleading on-chain data.

Recommendation

Fix the order of variable assignment and event parameters:

```
function setBounce(address _bounce) external onlyOwner {  
    if (_bounce == address(0)) revert ZeroAddress();  
  
    address oldBounce = bounce; //← capture current  
  
    bounce = _bounce; //← update to new  
  
    emit BounceUpdated(oldBounce, _bounce); //← correct order  
}
```

Status

Resolved

[Q-03] Contracts - Missing Event Emissions for Critical State Changes

Description

Several key state-changing operations across the Deadcat contracts modify critical variables but fail to emit events, making it difficult for off-chain services, analytics dashboards, or indexers to track important updates.

- **DeadcatBounce - Minimum Bounce Point Reset:** The `restart()` function reinitializes `minimumBouncePoint` without emitting an event. This change significantly affects gameplay logic and ticket distribution but cannot be tracked externally.
- **DedcatVault - Debt Balance Updates:** The vault records user debt changes in `_recordActivity()` without emitting any event. This makes it impossible to track user debt changes or reconcile vault balance adjustments from off-chain systems.
- **DeadcatCabal - Referral Chain Construction:** While the contract emits `ReferrerLinked` when linking a new referrer, there is no event emitted for full referral chain building, which limits visibility into referral hierarchy generation and tree updates.

Recommendation

Emit dedicated events for every critical state change that affects core protocol behavior.

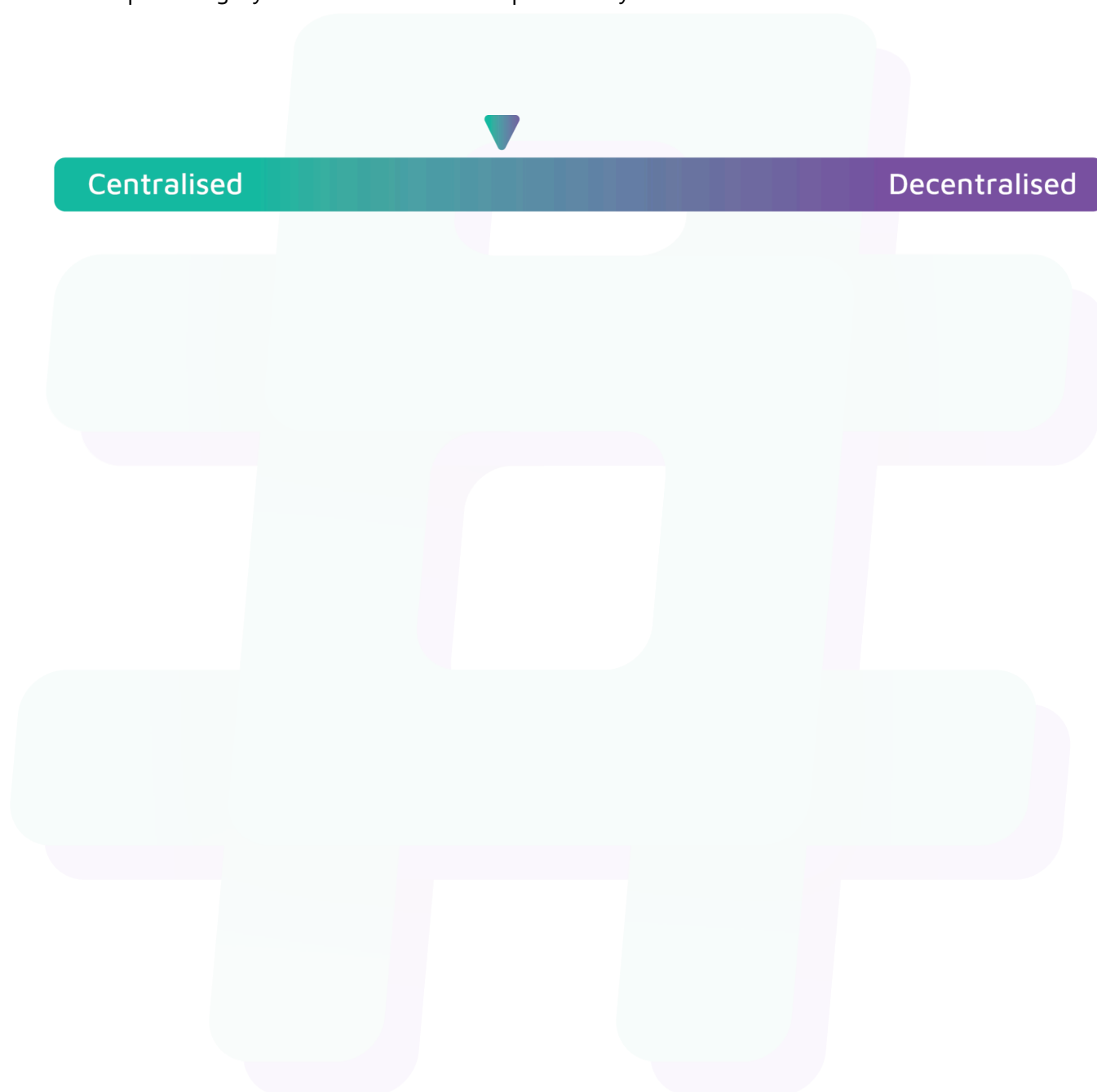
Status

Resolved

Centralisation

The DedcatBounce project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the DedcatBounce project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.